IAM-97: IAM Permission Reference — What Each Permission Actually Does

> **Series:** IAM — IAM Administration | **Reference:** 97 — IAM Permission Reference | **Created:** October 2026 | **Last Updated:** 10/19/2026

Overview

The IAM policy reference lists every permission with a one-line description, and most of those lines restate the permission's name. This appendix adds what the reference leaves out: what each permission really grants, its scope, when to grant it and when not to, its risk, and which built-in apps ask for it.

It covers every permission in Dynatrace's *Standard User* default policy, plus the permissions a reader persona and its neighbours usually need — Grail data, synthetic, credentials, settings writes and the admin variants.

Two rules shape every row:

- **The fact columns are generated** from the docs and a tenant, not typed, so they can be regenerated when Dynatrace changes a permission (see *Regenerating the Fact Columns*).
- **Every judgement shows its Basis** — Docs, Tested, Inference or Unknown. An inference is never presented as fact.

> **docs.dynatrace.com is authoritative.** Where this appendix and the docs disagree, the docs win.

Table of Contents

1. How to Read This Reference
2. Finding Out What a Permission Does
3. App state
4. Documents
5. Apps, functions and the UI shell
6. Grail definitions and segments
7. Grail data (record tables)
8. Settings and app settings
9. Email and notifications
10. Workflows (AutomationEngine)
11. Davis analyzers and Copilot
12. OpenPipeline
13. Synthetic monitoring
14. Credential vault
15. Security
16. Live Debugger
17. Classic roles
18. Other services
19. Persona Matrix
20. Open Questions
21. Regenerating the Fact Columns
22. References

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS on the latest Dynatrace platform with IAM policies
Prior Knowledge	**IAM-04** (policy syntax) and **IAM-05** (boundaries)
For the method section	`app-engine:apps:run` and `app-engine:functions:run`; optionally `dtctl`
For the effective-permissions API	An OAuth client with `iam:effective-permissions:read`

1. How to Read This Reference

Each service group has two tables. The first says what a permission is; the second says when to grant it.

Column	Source	Meaning
Std User	Generated	✅ in the docs *Standard User* default policy; ➕ added here for a persona
Reference says	Generated	The policy reference's own description, verbatim
Conditions	Generated	Condition keys and the operators each accepts. `none` means it cannot be scoped in a policy or boundary
What it really grants / Scope / Risk	Judgement	Scope is per user, per app, owner-based, per security context, or environment-wide
Basis	Judgement	Why we believe the judgement — see below
Allow when / Avoid when	Judgement	Grant guidance
Built-in apps that request it	Generated	Built-in apps whose manifest declares the permission, from the validation tenant
API operations that declare it	Generated	Platform API operations whose spec names the permission as the required scope

Basis	Means
Docs	A Dynatrace page says it. The row quotes the page (under 15 words) and links it
Tested	Observed on the validation tenant: an app manifest, a platform API spec, or behaviour. Dated
Inference	Our reasoning from documented facts. Not stated anywhere — treat as unconfirmed
Unknown	Neither the docs nor the tenant answer it. Test before you rely on it

A manifest entry shows that an app **may** call an API, not that every screen does: *"doesn't necessarily mean that the app uses it"*.

Generated from	Retrieved	Page last modified	Contents
IAM policy statements reference (DT docs)	10/19/2026	10/08/2026	43 services, 228 permissions
Default policies — Standard User (DT docs)	10/19/2026	10/08/2026	65 permissions
Built-in app manifests + platform API specs (validation tenant, via `dtctl`)	10/19/2026	—	118 built-in apps, 34 of 37 published APIs

This reference: 103 permissions — 65 from the docs *Standard User* policy (✅) and 38 reader-persona or companion permissions that are not in it (➕). **Basis:** 71 Docs, 24 Tested, 5 Inference, 3 Unknown.

> **Sources:** Add permission scopes (Dynatrace Developer); IAM policy reference (DT docs); Default policies (DT docs).

2. Finding Out What a Permission Does

When the reference doesn't explain a permission, five checks will. Use them in this order: the first two are read-only and fast; the last three confirm behaviour.

2.1 App manifest scopes

Every app declares the permissions it uses in its manifest, each with a short reason. At run time *"app permissions are intersected with user permissions"*, so a user gets an app's feature only when both hold the permission. Reading the manifests of the apps you allow is the most direct way to build a policy app by app.

Run this in a notebook **Code** section (or with `dtctl exec function -f`). It lists every permission each allowed app requests, with the app's own reason:

```
import { appEngineRegistryAppsClient } from "@dynatrace-sdk/client-app-engine-registry";

// Apps you allow (the shared:app-id values in your policy). Empty = every installed app.
const ALLOWED = ["dynatrace.dashboards", "dynatrace.notebooks", "dynatrace.davis.problems"];

export default async function () {
  const { apps } = await appEngineRegistryAppsClient.getApps({ addFields: "manifest.scopes" });
  const rows = [];
  for (const app of apps) {
    if (ALLOWED.length &amp;&amp; !ALLOWED.includes(app.id)) continue;
    for (const scope of app.manifest?.scopes ?? []) {
      rows.push({ app: app.id, permission: scope.name, reason: scope.comment ?? "" });
    }
  }
  return rows.sort((a, b) =&gt; a.permission.localeCompare(b.permission) || a.app.localeCompare(b.app));
}
```

Tested on the validation tenant (10/19/2026): it returns one row per app and permission. That is how this appendix found that the Problems app declares `openpipeline:events.davis:ingest` *"Write problem comments and close problems"*. From a shell, `dtctl get apps -o json --limit 0` returns the same manifests.

2.2 Platform API specs

Each platform API publishes an OpenAPI spec that names the required permission per operation. `dtctl describe api <name>` prints that index, for example:

```
dtctl describe api /platform/state-management/v1
```

This settles *what a permission unlocks* at endpoint level — `state-management:user-app-states:delete-all` is `DELETE /{appId}/user-app-states`, while the current-user variant is `DELETE /{appId}/user-app-states/self`. A blank scope means the spec declares none, not that none is needed: the email API's `POST /emails` declares nothing, yet `email:emails:send` exists for it.

2.3 The denied request in browser developer tools

Open the browser's developer tools, select **Network**, reproduce the failure, and filter for status `403`. The request path names the platform service and resource, and the spec from § 2.2 maps it to a permission. In community practice the response body often names the missing permission as well — check, as it varies by service.

2.4 A test user holding only the policy

Create a test group bound to only the draft policy and boundary, add one test user, then open every allowed app and use its main screens. Each failure is a 403 you can trace with § 2.3. Add only what is missing, and record the result as **Tested** evidence.

2.5 Effective policies and the effective-permissions API

Account Management → Identity & access management → **Effective policies** shows a user's combined result, to *"learn which permissions are assigned to users through policies and groups"*. The same check is an API:

```
GET https://api.dynatrace.com/iam/v1/resolution/account/{accountId}/effectivepermissions?entityType=user&amp;entityId={userId}
```

Two permissions are easy to confuse here:

Permission	Who it answers for	Used by
`iam:effective-permissions:read`	**Any** user or group in the account (`iam-param:entity-type` , `iam-param:entity-id`)	An OAuth client calling the account API above
`platform-management:effective-permissions:resolve`	**The calling user only**	Apps checking what the current user may do, to show or hide actions

> **Sources:**
>
> - Add permission scopes (Dynatrace Developer)
> - App Engine registry SDK (Dynatrace Developer)
> - Manage users, groups, and permissions (DT docs)
> - Upgrade role-based permissions to IAM (DT docs) — effective-permissions API path
> - Platform management SDK (Dynatrace Developer) — *"Get the effective permissions of the calling user"*
> - **Derived:** § 2.3 and § 2.4 combine the spec mapping in § 2.2 with standard browser and test-account practice

3. App state

Apps keep two kinds of state. **App state** is one key-value store per app, shared by every user of that app. **User app state** belongs to one user. The `state` service reads and writes them; the separate `state-management` service resets them in bulk — the Hub's reset action.

The trap in the docs *Standard User* policy is `state-management:user-app-states:delete-all`, which resets every user's state for an app. It, and `state:app-states:write` / `delete`, change the app for everyone.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`state:app-states:read`	✅	Grants permission to read app-states	`shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=)	Read the key-value state an app keeps for all of its users (caches, shared flags).	Per app, shared by every user	Low	**Docs** — *"internal global states or user-specific states"* (Store app and user states)
`state:app-states:write`	✅	Grants permission to write app-states	`shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=)	Change an app's shared state; the change is seen by every user of that app.	Per app, shared by every user	Medium	**Inference** — Follows from app states being shared (state service page); 18 built-in apps declare it, e.g. `dynatrace.discovery.coverage` *"write mute state"*.
`state:app-states:delete`	✅	Grants permission to delete app-states	`shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=)	Delete an app's shared state for every user of the app.	Per app, shared by every user	Medium	**Inference** — Same reasoning as write; the state service page shows a delete that *"completely reset[s] the app's states"*.
`state:user-app-states:read`	✅	Grants permission to read user-app-states	`shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=)	Read the caller's own per-app state (saved views, timeframes, dock favorites).	Per user, per app	Low	**Tested** — `dynatrace.dashboards` manifest: *"read view settings like the selected timeframe or variable values"* — validation tenant, 10/19/2026
`state:user-app-states:write`	✅	Grants permission to write user-app-states	`shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=)	Save the caller's own per-app state.	Per user, per app	Low	**Tested** — Declared by 35 built-in apps; `dynatrace.appshell` : *"persist UI state … across devices"* — validation tenant, 10/19/2026
`state:user-app-states:delete`	✅	Grants permission to delete user-app-states	`shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=)	Delete the caller's own per-app state.	Per user, per app	Low	**Docs** — *"Grants permission to delete user-app-states"* (IAM policy reference)
`state-management:user-app-states:delete`	✅	Grants permission to delete user-app-states of the current user	`shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=)	Reset *my* state for an app (the Hub reset action).	Per user, per app	Low	**Docs** — *"Grants permission to delete user-app-states of the current user"* (IAM policy reference)
`state-management:user-app-states:delete-all`	✅	Grants permission to delete user-app-states of all users	`shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=)	Reset every user's state for an app.	Environment-wide, per app	Medium	**Docs** — *"Grants*

permission to delete user-app-states of all users" (IAM policy reference) |
| `state-management:app-states:delete` | ➕ | Grants permission to delete all app-states | `shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=) | Reset an app's shared state (Admin User default, not Standard User). | Environment-wide, per app | Medium | **Docs** — *"Grants permission to delete all app-states"* (IAM policy reference) |

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare it *(generated)*
`state:app-states:read`	Always, when the app is allowed; scope with `shared:app-id`.	Never needed outside an allowed app.	`dynatrace.automations`, `dynatrace.azure.connector`, `dynatrace.biz.explore`, `dynatrace.discovery.coverage`, `dynatrace.experience.vitals`, `dynatrace.gitlab.connector`, +12 more	no published operation declares it
`state:app-states:write`	Admins, or users of an app whose manifest needs it for normal use.	Readers: a write changes the app for everyone.	`dynatrace.automations`, `dynatrace.azure.connector`, `dynatrace.biz.explore`, `dynatrace.discovery.coverage`, `dynatrace.experience.vitals`, `dynatrace.gitlab.connector`, +12 more	no published operation declares it
`state:app-states:delete`	Admins.	Readers and team leads.	`dynatrace.automations`, `dynatrace.azure.connector`, `dynatrace.biz.explore`, `dynatrace.microsoft.azure.connector`, `dynatrace.openpipeline`, `dynatrace.site.reliability.guardian`, +1 more	no published operation declares it
`state:user-app-states:read`	Always — most apps persist UI state here.	—	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, `dynatrace.davis.anomalydetection`, +28 more	no published operation declares it
`state:user-app-states:write`	Always.	—	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, `dynatrace.davis.anomalydetection`, +29 more	no published operation declares it
`state:user-app-states:delete`	Always.	—	`dynatrace.automations`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.distributedtracing`, `dynatrace.experience.vitals`, `dynatrace.kubernetes`, +8 more	no published operation declares it
`state-management:user-app-states:delete`	Optional for everyone.	—	`dynatrace.hub`	`DELETE /{appId}/user-app-states/self (/platform/state-management/v1)`
`state-management:user-app-states:delete-all`	Admins only.	Everyone else — it is in the docs Standard User policy, so remove it from copies of that policy.	`dynatrace.hub`	`DELETE /{appId}/user-app-states (/platform/state-management/v1)`
`state-management:app-states:delete`	Admins.	Everyone else.	`dynatrace.hub`	`DELETE /{appId}/app-states (/platform/state-management/v1)`

> **Sources:** Store app and user states (Dynatrace Developer) — *"internal global states or user-specific states"*; IAM policy reference (DT docs).

4. Documents

Dashboards, notebooks and launchpads are documents. Access follows ownership and shares, and no document permission takes an IAM condition. Two pairs matter for readers:

- `environment-shares:claim` without `environment-shares:write` lets a user **open** share links others create, but not create links that reach the whole environment.
- `trash.documents:restore` without `trash.documents:delete` keeps deletions recoverable.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`document:documents:read`	✅	Grants permission to read documents of the document service	none	Open dashboards, notebooks, launchpads and other documents the caller owns or that are shared with them.	Owner-based plus shares	Low	**Tested** — Document API: `GET /documents`, `GET /documents/{id}/content`; 19 built-in apps declare it — validation tenant, 10/19/2026
`document:documents:write`	✅	Grants permission to create and update documents of the document service	none	Create documents and edit ones the caller owns or has write shares on.	Owner-based plus shares	Low	**Inference** — Ownership limits it in practice; no IAM condition exists, so scope comes from document ownership and shares.
`document:documents:delete`	✅	Grants permission to delete documents of the document service	none	Move the caller's own documents to the trash.	Owner-based	Low	**Tested** — `dynatrace.appshell` manifest: *"move documents to the trash"* — validation tenant, 10/19/2026
`document:documents:admin`	➕	Grants admin permissions for documents of the document service	none	Act on documents regardless of owner (Admin User default).	Environment-wide	High	**Docs** — *"Grants admin permissions for documents of the document service"* (IAM policy reference)
`document:environment-shares:read`	✅	Grants permission to read environment shares of the document service	none	See the environment-wide share links that exist on a document.	Per document	Low	**Tested** — `dynatrace.appshell` manifest: *"show existing environment shares when managing document access"* — validation tenant, 10/19/2026
`document:environment-shares:write`	✅	Grants permission to create and update environment shares of the document service	none	Create a link that shares a document with every user in the environment.	Environment-wide reach	Medium	**Tested** — `dynatrace.appshell` manifest: *"share a document with all users in the environment"* — validation tenant, 10/19/2026
`document:environment-shares:claim`	✅	Grants permission to claim environment shares of the document service	none	Accept an environment share link so the document opens for the caller.	Per user	Low	**Tested** — `dynatrace.appshell` manifest: *"open a document accessed via an environment share link"*; API `PUT /environment-shares/{id}/claim` — validation tenant, 10/19/2026
`document:environment-shares:delete`	✅	Grants permission to delete environment shares of the document service	none	Remove an environment share link.	Per document	Low	**Docs** — *"Grants permission to delete environment shares of the document service"* (IAM policy reference)
`document:direct-shares:read`	✅	Grants permission to read direct shares of the document service	none	See which users or groups a document is shared with.	Per document	Low	**Docs** — *"Grants permission to read direct shares of the document service"* (IAM policy reference)
`document:direct-shares:write`	✅	Grants permission to create and update direct shares of the document service	none	Share a document with named users or groups.	Per document	Low	**Tested** — `dynatrace.appshell` manifest: *"share a document with specific users or groups"* — validation tenant, 10/19/2026
`document:direct-shares:delete`	✅	Grants permission to delete direct shares of the document service	none	Remove a direct share.	Per document	Low	**Docs** — *"Grants permission to delete direct shares of the document service"* (IAM policy reference)
`document:trash.documents:read`	✅	Grants permission to read deleted documents of the document service	none	See the caller's deleted documents.	Owner-based	Low	**Docs** — *"Grants permission to read deleted documents of the document service"* (IAM policy reference)
`document:trash.documents:restore`	✅	Grants permission to restore deleted documents from the trash of the document service	none	Restore a deleted document.	Owner-based	Low	**Docs** — *"Grants permission to restore deleted documents from the trash of the document service"* (IAM policy reference)
`document:trash.documents:delete`	✅	Grants permission to remove deleted documents from the trash of the document service	none	Permanently delete a trashed document before it expires.	Owner-based	Low	**Tested** — `dynatrace.notebooks` manifest: *"permanently delete notebooks"* — validation tenant, 10/19/2026

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that decla
`document:documents:read`	Always.	—	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, `dynatrace.davis.anomalydetection`, +13 more	`GET /documents` (/platform/document/v1 `GET /documents/{id}` (/platform/document/v1 `GET /documents/{id}/co` (/platform/document/v1 +7 more
`document:documents:write`	Anyone who should build dashboards or notebooks.	Strict read-only personas.	`dynatrace.appshell`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.davis.copilot`, `dynatrace.davis.problems`, `dynatrace.davis.workflow.actions`, +6 more	`DELETE /documents/{id}/snapsh version} (/platform/do `PATCH /documents/{id}` (/platform/document/v1 `PATCH /documents/{id}/` (/platform/document/v1 +8 more
`document:documents:delete`	With write.	Read-only personas.	`dynatrace.appshell`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.davis.copilot`, `dynatrace.davis.problems`, `dynatrace.davis.workflow.actions`, +6 more	`DELETE /documents/{id}` (/platform/document/v1 `POST /documents:delete` (/platform/document/v1
`document:documents:admin`	Admins.	Everyone else.	`dynatrace.documents`	no published operation dec
`document:environment-shares:read`	Always.	—	`dynatrace.appshell`, `dynatrace.dashboards`, `dynatrace.documents`, `dynatrace.launcher`, `dynatrace.notebooks`	`GET /environment-share` (/platform/document/v1 `GET /environment-share` (/platform/document/v1 `GET /environment-shares/{id}/claimers` (/platform/document/v1
`document:environment-shares:write`	Team leads and authors who publish content.	Readers — keeps them from publishing to everyone.	`dynatrace.appshell`, `dynatrace.documents`, `dynatrace.extensions.manager`	`POST /environment-shar` (/platform/document/v1
`document:environment-shares:claim`	Always — without it, share links fail to open.	—	`dynatrace.appshell`, `dynatrace.extensions.manager`	`PUT /environment-share` (/platform/document/v1
`document:environment-shares:delete`	With environment-shares:write.	Readers.	`dynatrace.appshell`, `dynatrace.documents`	`DELETE /environment-sh` (/platform/document/v1
`document:direct-shares:read`	Always.	—	`dynatrace.appshell`, `dynatrace.dashboards`, `dynatrace.documents`, `dynatrace.launcher`, `dynatrace.notebooks`	`GET /direct-shares` (/platform/document/v1 `GET /direct-shares/{id` (/platform/document/v1 `GET /direct-shares/{id` (/platform/document/v1

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that decla
`document:direct-shares:write`	Anyone who owns documents.	—	`dynatrace.appshell`, `dynatrace.documents`	`POST /direct-shares` (/platform/document/v1 `POST /direct-shares/{id}/recipients` (/platform/document/v1 `POST /direct-shares/{id}/recipients` (/platform/document/v1
`document:direct-shares:delete`	With direct-shares:write.	—	`dynatrace.appshell`, `dynatrace.documents`	`DELETE /direct-shares/` (/platform/document/v1
`document:trash.documents:read`	Always.	—	`dynatrace.dashboards`, `dynatrace.documents`, `dynatrace.launcher`, `dynatrace.notebooks`	`GET /trash/documents` (/platform/document/v1 `GET /trash/documents/{` (/platform/document/v1
`document:trash.documents:restore`	Always.	—	`dynatrace.dashboards`, `dynatrace.documents`, `dynatrace.launcher`, `dynatrace.notebooks`	`POST /trash/documents/` (/platform/document/v1
`document:trash.documents:delete`	Authors.	Readers — keeps deletions recoverable.	`dynatrace.dashboards`, `dynatrace.documents`, `dynatrace.launcher`, `dynatrace.notebooks`	`DELETE /trash/document` (/platform/document/v1

> **Sources:** IAM policy reference (DT docs).

5. Apps, functions and the UI shell

`app-engine:apps:run` is the gate for every app — scope it to an allow-list with `shared:app-id`. Most other rows here are platform calls the UI makes on the user's behalf, so removing them breaks screens rather than closing a data path.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`app-engine:apps:run`	✅	Grants permission to list and run apps and gives basic access to the Launcher	`shared:app-id` (IN, =, !=, startsWith, NOT IN, NOT startsWith) `app-engine:app-installer` (IN, =, !=, startsWith, NOT IN, NOT startsWith)	List and launch apps; the Launcher needs it. Also gates app functions.	Per app (`shared:app-id`)	Low	**Docs** — *"Grants permission to list and run apps and gives basic access to the Launcher"* (IAM policy reference)
`app-engine:functions:run`	✅	Grants permission to use the function-executor	none	Run code through the function executor (ad-hoc functions in notebooks, dashboards and workflows).	Environment-wide	Low	**Docs** — *"Grants permission to use the function-executor"* (IAM policy reference)
`app-engine:edge-connects:read`	✅	Grants permission to read EdgeConnects	none	List EdgeConnect definitions (host patterns and routing).	Environment-wide	Low	**Unknown** — Whether apps or workflows that send traffic through an EdgeConnect fail without it is not documented; `dynatrace.dashboards` and `dynatrace.notebooks` declare it *"via functions"*.
`unified-analysis:screen-definition:read`	✅	Grants permission to read the screen definition of a unified analysis screen	none	Read the screen definitions behind entity details panels (incl. extension screens).	Environment-wide	Low	**Tested** — 10 built-in apps declare it, e.g. `dynatrace.clouds`: *"Necessary for displaying details panel"* — validation tenant, 10/19/2026
`hub:catalog:read`	✅	Grants permission to read the hub catalog content.	none	Browse the Hub catalog and app release notes. Installing needs separate permissions.	Environment-wide	Low	**Docs** — *"Grants permission to read the hub catalog content."* (IAM policy reference)
`platform-management:effective-permissions:resolve`	✅	Grants permission to read operations of the public effective-permissions PMS API endpoint	none	Let the calling user ask *"which of these permissions do I have?"* — for their own identity only.	Per user (self)		

| Low | **Docs** — *"Get the effective permissions of the calling user"* (Platform management SDK) |
| `platform-management:environments:read` | ✅ | Grants permission to read operations of the environment data of the public environment PMS API | none | Read environment metadata and license information. | Environment-wide | Low | **Tested** — Management API: `GET /environment`, `GET /environment/license` — validation tenant, 10/19/2026 |

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare it *(generated)*
`app-engine:apps:run`	Always — scope to an allow-list of app IDs.	Unscoped grants: every installed app becomes usable.	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.dashboards`, `dynatrace.experience.vitals`, `dynatrace.fleet.management`, `dynatrace.hub`, +6 more	`GET /apps/{appId}/api/{functionN (/platform/app-engine/app-functions/v1)` `POST /apps/{appId}/api/{functionN (/platform/app-engine/app-functions/v1)` `GET /apps (/platform/app-engine/registry/v1)` +7 more
`app-engine:functions:run`	Almost always — 22 built-in apps request it.	Locked-down kiosk users.	`dynatrace.automations`, `dynatrace.aws.connector`, `dynatrace.azure.connector`, `dynatrace.dashboards`, `dynatrace.davis.workflow.actions`, `dynatrace.gitlab.connector`, +16 more	`GET /sdk-versions (/platform/app-engine/functi executor/v1)` `POST /executions (/platform engine/function-executor/v1)` `GET /environment (/platform/management/v1)` +4 more
`app-engine:edge-connects:read`	Admins and authors who route requests through an EdgeConnect.	Readers — nothing in a reader's normal path declares a need.	`dynatrace.dashboards`, `dynatrace.notebooks`, `dynatrace.outbound.connections`	`GET /edge-connects (/platform/app-engine/edge-connect/v1)` `GET /edge-connects/{edgeConnectId} (/platform/app-engine/edge-connect/v1)` `GET /edge-connects:match (/platform/app-engine/edge-connect/v1)`
`unified-analysis:screen-definition:read`	Readers of infrastructure, cloud, database and Kubernetes apps.	—	`dynatrace.clouds`, `dynatrace.database.overview`, `dynatrace.error.inspector`, `dynatrace.fleet.management`, `dynatrace.infraops`, `dynatrace.kubernetes`, +4 more	no published operation declares it
`hub:catalog:read`	Always.	—	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.davis.copilot`, `dynatrace.discovery.coverage`, +13 more	`GET /catalog/apps (/platform/hub/v1)` `GET /catalog/apps/{id} (/platform/hub/v1)` `GET /catalog/apps/{id}/rele (/platform/hub/v1)` +6 more
`platform-management:effective-permissions:resolve`	Always — apps use it to show or hide actions.	—	none declare it	`POST /effective-permissions:resolve (/platform/management/v1)`

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare it *(generated)*
`platform-management:environments:read`	Always.	—	none declare it	`GET /environment` (/platform/management/v1) `GET /environment/license` (/platform/management/v1) `GET /environment/license/setting` (/platform/management/v1) +1 more

> **Sources:** IAM policy reference (DT docs); Platform management SDK (Dynatrace Developer).

6. Grail definitions and segments

Definitions describe *where* data lives and how it is filtered; they return no records. Segments are filters, not access control — a segment never widens or narrows what a user can read.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`storage:bucket-definitions:read`	✅	Grants permission to read bucket definitions from Grail	none	List bucket names and settings — not their records.	Environment-wide	Low	**Docs** — *"Grants permission to read bucket definitions from Grail"* (IAM policy reference)
`storage:fieldset-definitions:read`	✅	Read fieldset definitions from grail	none	Read fieldset definitions — which fields are protected.	Environment-wide	Low	**Tested** — `dynatrace.davis.copilot` manifest: *"query masked/sensitive fields"* — validation tenant, 10/19/2026
`storage:filter-segments:read`	✅	Read filter-segments from grail	none	Use segments in segment pickers.	Environment-wide	Low	**Docs** — *"Read filter-segments from grail"* (IAM policy reference)
`storage:filter-segments:write`	✅	Write filter-segments in grail	none	Create and edit segments.	Environment-wide	Low	**Docs** — *"Write filter-segments in grail"* (IAM policy reference)
`storage:filter-segments:delete`	✅	Delete own filter-segments in grail	none	Delete the caller's own segments.	Owner-based	Low	**Docs** — *"Delete own filter-segments in grail"* (IAM policy reference)
`storage:filter-segments:share`	➕	Share filter-segments in grail	none	Share a segment with others.	Owner-based	Low	**Docs** — *"Share filter-segments in grail"* (IAM policy reference)
`storage:filter-segments:admin`	➕	Write and delete all filter-segments in grail	none	Edit and delete any user's segments.	Environment-wide	Medium	**Docs** — *"Write and delete all filter-segments in grail"* (IAM policy reference)

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare it *(generated)*
`storage:bucket-definitions:read`	Always; bucket pickers use it.	—	`dynatrace.automations`, `dynatrace.dashboards`, `dynatrace.davis.copilot`, `dynatrace.davis.workflow.actions`, `dynatrace.dpl.architect`, `dynatrace.experience.vitals`, +4 more	no published operation declares it
`storage:fieldset-definitions:read`	Always.	—	`dynatrace.davis.copilot`, `dynatrace.davis.workflow.actions`, `dynatrace.experience.vitals`, `dynatrace.openpipeline`	`GET /fieldsets` (/platform/storage/fieldsets/v1) `GET /fieldsets/{fieldsetUid}` (/platform/storage/fieldsets/v1)

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare it *(generated)*
`storage:filter-segments:read`	Always.	—	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.biz.explore`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, +25 more	no published operation declares it
`storage:filter-segments:write`	Team leads who curate segments.	Readers, if segments are centrally managed.	`dynatrace.segments.management`	no published operation declares it
`storage:filter-segments:delete`	With write.	Readers.	`dynatrace.segments.management`	no published operation declares it
`storage:filter-segments:share`	Team leads.	Readers.	`dynatrace.segments.management`	no published operation declares it
`storage:filter-segments:admin`	Admins.	Everyone else.	`dynatrace.segments.management`	no published operation declares it

> **Sources:** IAM policy reference (DT docs).

7. Grail data (record tables)

These are **not** in the docs *Standard User* policy; a reader persona needs some of them to see any data. Each table permission also needs `storage:buckets:read`. Scope them in the **boundary**, and use `MATCH` for security context on tables where it can be an array — `=`, `IN` and `startsWith` return false on arrays. `storage:entities:read` is the exception: it accepts no `MATCH`, so pair a `startsWith` line with the `MATCH` line.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`storage:buckets:read`	➕	Grants permission to read records from Grail buckets. Required additionally to a table permission.	`storage:table-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH) `storage:bucket-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH) `storage:query-consumption` (=)	Read records from buckets; needed in addition to every table permission.	Per bucket (`storage:bucket-name`)	Depends on the buckets	**Docs** — *"Required additionally to a table permission."* (IAM policy reference)
`storage:logs:read`	➕	Grants permission to read records from the logs-table	`storage:bucket-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH) `storage:k8s.namespace.name` (=, IN, startsWith, MATCH) `storage:k8s.cluster.name` (=, IN, startsWith, MATCH) `storage:host.name` (=, IN, startsWith, MATCH) `storage:dt.host_group.id` (=, IN, startsWith, MATCH) `storage:log.source` (=, IN, startsWith, MATCH) `storage:dt.security_context` (=, IN, startsWith, MATCH) `storage:gcp.project.id` (=, IN, startsWith, MATCH) `storage:aws.account.id` (=, IN, startsWith, MATCH) `storage:azure.subscription` (=, IN, startsWith, MATCH) `storage:azure.resource.group` (=, IN, startsWith, MATCH)	Query log records.	Records filtered by conditions	Medium — logs can carry PII	**Docs** — *"Grants permission to read records from the logs-table"* (IAM policy reference)
`storage:spans:read`	➕	Grants permission to read records from the spans-table	`storage:bucket-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH) `storage:k8s.namespace.name` (=, IN, startsWith, MATCH) `storage:k8s.cluster.name` (=, IN, startsWith, MATCH)				

`storage:host.name` (=, IN, startsWith, MATCH)
`storage:dt.host_group.id` (=, IN, startsWith, MATCH)
`storage:dt.security_context` (=, IN, startsWith, MATCH)
`storage:gcp.project.id` (=, IN, startsWith, MATCH)
`storage:aws.account.id` (=, IN, startsWith, MATCH)
`storage:azure.subscription` (=, IN, startsWith, MATCH)
`storage:azure.resource.group` (=, IN, startsWith, MATCH) | Query spans (distributed traces). | Records filtered by conditions | Medium | **Docs** — *"Grants permission to read records from the spans-table"* (IAM policy reference) |
| `storage:metrics:read` | ➕ | Grants permission to read timeseries from the metrics-table | `storage:bucket-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH)
`storage:k8s.namespace.name` (=, IN, startsWith, MATCH)
`storage:k8s.cluster.name` (=, IN, startsWith, MATCH)
`storage:host.name` (=, IN, startsWith, MATCH)
`storage:dt.host_group.id` (=, IN, startsWith, MATCH)
`storage:metric.key` (=, IN, startsWith, MATCH)
`storage:dt.security_context` (=, IN, startsWith, MATCH)
`storage:gcp.project.id` (=, IN, startsWith, MATCH)
`storage:aws.account.id` (=, IN, startsWith, MATCH)
`storage:azure.subscription` (=, IN, startsWith, MATCH)
`storage:azure.resource.group` (=, IN, startsWith, MATCH)
`storage:frontend.name` (=, IN, startsWith, MATCH) | Query metric timeseries. | Records filtered by conditions | Low | **Docs** — *"Grants permission to read timeseries from the metrics-table"* (IAM policy reference) |
| `storage:events:read` | ➕ | Grants permission to read records from the events table. | `storage:bucket-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH)
`storage:event.kind` (=, IN, startsWith, MATCH)
`storage:event.type` (=, IN, startsWith, MATCH)
`storage:event.provider` (=, IN, startsWith, MATCH)
`storage:k8s.namespace.name` (=, IN, startsWith, MATCH)
`storage:k8s.cluster.name` (=, IN, startsWith, MATCH)
`storage:host.name` (=, IN, startsWith, MATCH)
`storage:dt.host_group.id` (=, IN, startsWith, MATCH)
`storage:dt.security_context` (=, IN, startsWith, MATCH)
`storage:gcp.project.id` (=, IN, startsWith, MATCH)
`storage:aws.account.id` (=, IN, startsWith, MATCH)
`storage:azure.subscription` (=, IN, startsWith, MATCH)
`storage:azure.resource.group` (=, IN, startsWith, MATCH)
`storage:frontend.name` (=, IN, startsWith, MATCH) | Query events, including Davis problems and events. | Records filtered by conditions | Low | **Docs** — *"always return false"* (Permissions in Grail) |
| `storage:bizevents:read` | ➕ | Grants permission to read records from the bizevents-table | `storage:bucket-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH)
`storage:event.kind` (=, IN, startsWith, MATCH)
`storage:event.type` (=, IN, startsWith, MATCH)
`storage:event.provider` (=, IN, startsWith, MATCH)
`storage:k8s.namespace.name` (=, IN, startsWith, MATCH)
`storage:k8s.cluster.name` (=, IN, startsWith, MATCH)
`storage:host.name` (=, IN, startsWith, MATCH)
`storage:dt.host_group.id` (=, IN, startsWith, MATCH)
`storage:dt.security_context` (=, IN, startsWith, MATCH)
`storage:gcp.project.id` (=, IN, startsWith, MATCH)
`storage:aws.account.id` (=, IN, startsWith, MATCH)
`storage:azure.subscription` (=, IN, startsWith, MATCH)
`storage:azure.resource.group` (=, IN, startsWith, MATCH)
`storage:frontend.name` (=, IN, startsWith, MATCH) | Query business events. | Records filtered by conditions | Medium | **Docs** — *"Grants permission to read records from the bizevents-table"* (IAM policy reference) |
| `storage:entities:read` | ➕ | Grants permission to read records from entities | `storage:entity.type` (=, IN, startsWith)
`storage:dt.security_context` (=, IN, startsWith) | Query classic entities (`fetch dt.entity.*`). | Records filtered by conditions | Low | **Docs** — *"Grants permission to read records from entities"* (IAM policy reference) |
| `storage:smartscape:read` | ➕ | Grants permission to read smartscape nodes and edges from Grail | `storage:dt.security_context` (=, IN, startsWith, MATCH)

`storage:k8s.namespace.name` (=, IN, startsWith, MATCH)
`storage:k8s.cluster.name` (=, IN, startsWith, MATCH)
`storage:host.name` (=, IN, startsWith, MATCH)
`storage:dt.host_group.id` (=, IN, startsWith, MATCH)
`storage:gcp.project.id` (=, IN, startsWith, MATCH)
`storage:aws.account.id` (=, IN, startsWith, MATCH)
`storage:azure.subscription` (=, IN, startsWith, MATCH)
`storage:azure.resource.group` (=, IN, startsWith, MATCH)
`storage:frontend.name` (=, IN, startsWith, MATCH) | Query Smartscape nodes and edges (`smartscapeNodes`). | Records filtered by conditions | Low | **Docs** — *"Grants permission to read smartscape nodes and edges from Grail"* (IAM policy reference) |
| `storage:system:read` | ➕ | Grants permission to read records from all system tables (for example, `dt.system.events`). |
`storage:bucket-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH)
`storage:event.kind` (=, IN, startsWith, MATCH)
`storage:event.type` (=, IN, startsWith, MATCH)
`storage:event.provider` (=, IN, startsWith, MATCH)
`storage:dt.security_context` (=, IN, startsWith, MATCH) | Query every system table, e.g. `dt.system.events` (audit and billing). | Environment-wide tables | High | **Docs** — *"Grants permission to read records from all system tables"* (IAM policy reference) |
| `storage:files:read` | ➕ | Read data from files. | `storage:file-path` (=, startsWith, IN) | Read files such as lookup tables. | Per path (`storage:file-path`) | Low | **Docs** — *"storage:files:read WHERE storage:file-path startsWith "/dt/""* (Default policies) |
| `storage:user.events:read` | ➕ | Read user.events from grail | `storage:bucket-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH)
`storage:dt.security_context` (=, IN, startsWith, MATCH)
`storage:frontend.name` (=, IN, startsWith, MATCH) | Query RUM user events (needed by Experience Vitals). | Records filtered by conditions | Medium | **Tested** — `dynatrace.experience.vitals` and 25 other built-in apps declare it — validation tenant, 10/19/2026 |
| `storage:user.sessions:read` | ➕ | Read user.sessions from grail | `storage:bucket-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH)
`storage:dt.security_context` (=, IN, startsWith, MATCH)
`storage:frontend.name` (=, IN, startsWith, MATCH) | Query RUM user sessions. | Records filtered by conditions | Medium | **Docs** — *"Read user.sessions from grail"* (IAM policy reference) |
| `storage:security.events:read` | ➕ | Read security.events from grail | `storage:bucket-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH)
`storage:event.kind` (=, IN, startsWith, MATCH)
`storage:event.type` (=, IN, startsWith, MATCH)
`storage:event.provider` (=, IN, startsWith, MATCH)
`storage:k8s.namespace.name` (=, IN, startsWith, MATCH)
`storage:k8s.cluster.name` (=, IN, startsWith, MATCH)
`storage:host.name` (=, IN, startsWith, MATCH)
`storage:dt.host_group.id` (=, IN, startsWith, MATCH)
`storage:dt.security_context` (=, IN, startsWith, MATCH)
`storage:gcp.project.id` (=, IN, startsWith, MATCH)
`storage:aws.account.id` (=, IN, startsWith, MATCH)
`storage:azure.subscription` (=, IN, startsWith, MATCH)
`storage:azure.resource.group` (=, IN, startsWith, MATCH) | Query security events (vulnerability, detection and compliance findings). | Records filtered by conditions | High | **Docs** — *"Read security.events from grail"* (IAM policy reference) |
| `storage:application.snapshots:read` | ➕ | Read application.snapshots from grail | `storage:bucket-name` (=, !=, IN, NOT IN, startsWith, NOT startsWith, MATCH)
`storage:k8s.namespace.name` (=, IN, startsWith, MATCH)
`storage:k8s.cluster.name` (=, IN, startsWith, MATCH)
`storage:host.name` (=, IN, startsWith, MATCH)
`storage:dt.host_group.id` (=, IN, startsWith, MATCH)
`storage:dt.security_context` (=, IN, startsWith, MATCH)
`storage:gcp.project.id` (=, IN, startsWith, MATCH)
`storage:aws.account.id` (=, IN, startsWith, MATCH)
`storage:azure.subscription` (=, IN, startsWith, MATCH)
`storage:azure.resource.group` (=, IN, startsWith, MATCH) | Read Live Debugger snapshots, which hold variable values from running code. | Records filtered by conditions | High — runtime data may include secrets or PII | **Docs** — *"stack trace, variable values, and more"* (Live Debugger) |

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	Al *(g*
`storage:buckets:read`	Always, with a table permission.	—	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.biz.explore`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, +35 more	`G` `(/`
`storage:logs:read`	Readers of this signal; scope with the boundary (`storage:dt.security_context` with `MATCH`).	Unscoped grants for team users.	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, `dynatrace.davis.anomalydetection`, +30 more	no
`storage:spans:read`	Readers of this signal; scope with the boundary (`storage:dt.security_context` with `MATCH`).	Unscoped grants for team users.	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, `dynatrace.davis.anomalydetection`, +23 more	no
`storage:metrics:read`	Readers of this signal; scope with the boundary (`storage:dt.security_context` with `MATCH`).	—	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, `dynatrace.davis.anomalydetection`, +31 more	no
`storage:events:read`	Readers of this signal; scope with the boundary (`storage:dt.security_context` with `MATCH`). Events often store the security context as an array.	—	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.biz.explore`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, +26 more	no
`storage:bizevents:read`	Teams that use business events.	Users without a need — payloads can hold business data.	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.biz.explore`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.davis.anomalydetection`, +21 more	no
`storage:entities:read`	Readers; the security-context condition takes `=`, `IN`, `startsWith` only — not `MATCH`.	—	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, `dynatrace.davis.anomalydetection`, +28 more	no

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	Al *(g*
`storage:smartscape:read`	Readers of this signal; scope with the boundary (`storage:dt.security_context` with `MATCH`).	—	`dynatrace.appshell` , `dynatrace.automations` , `dynatrace.biz.explore` , `dynatrace.clouds` , `dynatrace.dashboards` , `dynatrace.database.overview` , +31 more	nc
`storage:system:read`	Admins and cost owners.	Readers — audit and consumption records are not team data.	`dynatrace.appshell` , `dynatrace.automations` , `dynatrace.clouds` , `dynatrace.dashboards` , `dynatrace.database.overview` , `dynatrace.davis.anomalydetection` , +17 more	nc
`storage:files:read`	Scope to your lookup folders plus `/dt/` , as Dynatrace's *All Grail data read access* policy does.	Unscoped grants.	`dynatrace.appshell` , `dynatrace.automations` , `dynatrace.clouds` , `dynatrace.dashboards` , `dynatrace.davis.anomalydetection` , `dynatrace.davis.copilot` , +19 more	nc
`storage:user.events:read`	Frontend teams; scope by `frontend.name` or security context.	Teams without RUM.	`dynatrace.appshell` , `dynatrace.automations` , `dynatrace.clouds` , `dynatrace.dashboards` , `dynatrace.davis.anomalydetection` , `dynatrace.davis.copilot` , +20 more	`G` `(/`
`storage:user.sessions:read`	With user.events.	Teams without RUM.	`dynatrace.appshell` , `dynatrace.automations` , `dynatrace.clouds` , `dynatrace.dashboards` , `dynatrace.davis.anomalydetection` , `dynatrace.davis.copilot` , +21 more	nc
`storage:security.events:read`	Security teams.	General readers.	`dynatrace.appshell` , `dynatrace.automations` , `dynatrace.clouds` , `dynatrace.dashboards` , `dynatrace.davis.anomalydetection` , `dynatrace.davis.copilot` , +21 more	nc
`storage:application.snapshots:read`	Only with Live Debugger, via the *Read Application Snapshots* default policy.	General Grail read policies.	`dynatrace.automations` , `dynatrace.dashboards` , `dynatrace.davis.anomalydetection` , `dynatrace.davis.copilot` , `dynatrace.davis.workflow.actions` , `dynatrace.dpl.architect` , +2 more	nc

> **Sources:** Permissions in Grail (DT docs) — *"Using =, STARTSWITH or IN when the field holds an array will always return false."*; Default policies (DT docs); IAM policy reference (DT docs).

8. Settings and app settings

Three stores are easy to confuse:

Store	Permissions	Holds	Scope
Settings 2.0	`settings:objects:*`	Monitoring configuration and user preferences	Environment, host, entity or `user-` scope
App settings	`app-settings:objects:*`	Objects of schemas an app defines (e.g. connector connections)	Per app schema; some schemas are owner-based
App state	`state:*`	An app's internal key-value data	Per app, or per user

Owner-based schemas give each object to the user who first stores it; `*:objects:admin` bypasses that check. For settings scoping, use `settings:dt.security_context` rather than `environment:management-zone`, which stops matching after the scoped-settings change.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`settings:objects:read`	✅	Enables reading of settings objects belonging to the schema	`settings:schemaId` (IN, =, !=, startsWith, NOT startsWith) `shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=) `settings:schemaGroup` (IN, =) `settings:entity.hostGroup` (IN, =, !=) `settings:scope` (IN, =, !=, startsWith, NOT startsWith) `environment:management-zone` (IN, =, startsWith, MATCH) `settings:dt.security_context` (IN, =, startsWith)	Read Settings 2.0 objects (environment, host, entity and user scopes).	Per schema / scope / security context	Medium	**Docs** — *"Enables reading of settings objects belonging to the schema"* (IAM policy reference)
`settings:schemas:read`	✅	Enables reading settings schemas	`settings:schemaId` (IN, =, !=, startsWith, NOT startsWith) `shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=) `settings:schemaGroup` (IN, =)	Read settings schema definitions — not values.	Per schema	Low	**Docs** — *"Enables reading settings schemas"* (IAM policy reference)
`settings:objects:write`	➕	Enables writing of settings objects belonging to the schema	`settings:schemaId` (IN, =, !=, startsWith, NOT startsWith) `shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=) `settings:schemaGroup` (IN, =) `settings:entity.hostGroup` (IN, =, !=) `settings:scope` (IN, =, !=, startsWith, NOT startsWith) `environment:management-zone` (IN, =, startsWith, MATCH) `settings:dt.security_context` (IN, =, startsWith)	Change settings objects. With `settings:scope startsWith 'user-'` it covers only the user's own preferences.	Per schema / scope	High when unscoped	**Docs** — *"Enables writing of settings objects belonging to the schema"* (IAM policy reference)
`settings:objects:admin`	➕	Enables using admin-mode to access, change ownership and share permissions of any object. Admin-mode only bypasses the ownership check - so to do anything useful, settings:objects:read and/or settings:objects:write are needed as well.	`settings:schemaId` (IN, =, !=, startsWith, NOT startsWith) `shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=) `settings:schemaGroup` (IN, =)	Bypass the ownership check on owner-based settings objects.	Environment-wide	High	**Docs** — *"Admin-mode only bypasses the ownership check"* (IAM policy reference)
`app-settings:objects:read`	✅	Grants permission to read app settings objects belonging to the schema	`settings:schemaId` (IN, =, !=, startsWith, NOT startsWith) `shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=)	Read objects of app-defined schemas, such as workflow connector connections.	Per app schema; some schemas are owner-based	Low to medium	**Docs** — *"All authenticated users of the app can read app settings."* (Store app settings)
`app-settings:objects:write`	➕	Grants permission to write settings objects belonging to the schema	`settings:schemaId` (IN, =, !=, startsWith, NOT startsWith) `shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=)	Create or change app-schema objects. For most schemas, the change affects every user; owner-based schemas limit it to the owner.	Per app schema	Medium	**Docs** — *"the user that first stores a settings object is the owner"* (Store app settings)
`app-settings:objects:admin`	➕	Enables using admin-mode to access, change ownership and share permissions of any object. Admin-mode only bypasses the ownership check - so to do anything useful, app-settings:objects:read and/or app-settings:objects:write are needed as well.	`settings:schemaId` (IN, =, !=, startsWith, NOT startsWith) `shared:app-id` (IN, NOT IN, startsWith, NOT startsWith, =, !=)	Bypass the ownership check on owner-based app-schema objects (e.g.			

read others' connector credentials). | Environment-wide | High | **Tested** — `dynatrace.jenkins.connector` manifest: *"Reading and editing Connection credentials … as an admin"* — validation tenant, 10/19/2026 |

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations *(generated)*
`settings:objects:read`	Scope by `settings:schemaId`; add a `settings:dt.security_context` line to the boundary.	Unscoped grants plus a boundary with no settings line.	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.aws.connector`, `dynatrace.biz.explore`, `dynatrace.clouds`, `dynatrace.dashboards`, +40 more	`GET /classic pipelines/tra (/platform/op` *(early adopter)* `GET /effecti (/platform/se` `GET /history (/platform/se` +5 more
`settings:schemas:read`	Always.	—	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.biz.explore`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.davis.anomalydetection`, +25 more	`GET /schemas (/platform/se` `GET /schemas (/platform/se` `GET /values/ id}/default (/platform/se` +4 more
`settings:objects:write`	Everyone, limited to user-scope preference schemas; team leads, limited to their schemas.	Unscoped grants.	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.aws.connector`, `dynatrace.biz.explore`, `dynatrace.clouds`, `dynatrace.clouds`, +29 more	`DELETE /obje (/platform/se` `DELETE /obje id}/permissio (/platform/se` `DELETE /obje id}/permissio type}/{access (/platform/se` +7 more
`settings:objects:admin`	Admins.	Everyone else.	`dynatrace.aws.connector`, `dynatrace.fleet.management`, `dynatrace.gcp.connector`, `dynatrace.microsoft.azure.connector`, `dynatrace.openpipeline`, `dynatrace.security.enrichment`, +2 more	no published op it
`app-settings:objects:read`	When an allowed app needs it; scope by `shared:app-id` or `settings:schemaId`.	—	`dynatrace.appshell`, `dynatrace.azure.connector`, `dynatrace.biz.explore`, `dynatrace.clouds`, `dynatrace.database.overview`, `dynatrace.devobs.debugger`, +22 more	no published op it
`app-settings:objects:write`	Workflow authors (connections) and app owners.	Readers.	`dynatrace.azure.connector`, `dynatrace.clouds`, `dynatrace.database.overview`, `dynatrace.discovery.coverage`, `dynatrace.github.connector`, `dynatrace.gitlab.connector`, +16 more	no published op it
`app-settings:objects:admin`	Admins.	Everyone else.	`dynatrace.azure.connector`, `dynatrace.github.connector`, `dynatrace.gitlab.connector`, `dynatrace.jenkins.connector`, `dynatrace.jira`, `dynatrace.jsm.connector`, +8 more	no published op it

> **Sources:** Store app settings (Dynatrace Developer) — *"By default, only the owner can access these objects"*; Upgrade security policies for scoped settings (DT docs); IAM policy reference (DT docs).

9. Email and notifications

`email:emails:send` takes no conditions, so it can mail any address.

Notification API v1 is being retired. The self-notification permissions call it, and Dynatrace states it was *"deprecated on 9 Jul 2025 and will be sunset on 31 Dec 2026"*, pointing to the v2 event notifications that `notification:notifications:*` govern. Both rows stay in the *Standard User* policy for now; plan new work on v2.

The generated *API operations* column flags any API whose own spec on the tenant states a deprecation (⚠️), and any operation the spec marks pre-GA, such as *(early adopter)*. Workflows send as their **actor**, so the actor needs it — not every author.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`email:emails:send`	✅	Grants permission to send emails from @apps.dynatrace.com with send email API	none	Send email from `@apps.dynatrace.com` through the email API; no IAM condition limits the recipients.	Environment-wide, no conditions	Medium — can mail anyone	**Docs** — *"send emails from @apps.dynatrace.com with send email API"* (IAM policy reference)
`notification:self-notifications:read`	✅	Grants permission to read self notifications.	none	Read the caller's own notification subscriptions through Notification API v1 — deprecated, and sunset on 31 Dec 2026.	Per user	Low	**Docs** — *"deprecated on 9 Jul 2025 and will be sunset on 31 Dec 2026"* (APIs for latest Dynatrace)
`notification:self-notifications:write`	✅	Grants permission to write self notifications.	none	Create and change the caller's own subscriptions (Notification API v1, sunset 31 Dec 2026).	Per user	Low	**Tested** — Notification API v1: `POST /self-notifications`; whether the permission keeps a use after the v1 sunset is not documented — validation tenant, 10/19/2026
`notification:notifications:read`	✅	Grants permission to read notification configurations.	none	Read event- and resource-notification configurations, e.g. alerting on saved Problems filters.	Environment-wide	Low	**Tested** — `dynatrace.davis.problems` manifest: *"Read notifications for alerting on saved filters"* — validation tenant, 10/19/2026
`notification:notifications:write`	✅	Grants permission to write notification configurations.	none	Create and change those configurations.	Environment-wide	Medium	**Unknown** — Whether a user can change notifications created by others is not documented; the API has no owner filter in its declared scopes.

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operation
`email:emails:send`	Workflow actors (ideally service users).	Readers — the docs Standard User policy includes it.	`dynatrace.automations`, `dynatrace.email`, `dynatrace.hub`	no published o
`notification:self-notifications:read`	When users subscribe to alerts for themselves.	Building anything new on it: Dynatrace points to the v2 event notifications, which `notification:notifications:*` govern.	none declare it	`GET /self-n` (/platform/n `GET /self-n` (/platform/n ⚠️ `/platfor` 2025, sunset 3
`notification:self-notifications:write`	With read.	New automation — use v2 event notifications instead.	none declare it	`DELETE /sel` (/platform/n `PATCH /self` (/platform/n `POST /self-` (/platform/n +1 more ⚠️ `/platfor` 2025, sunset 3

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operation
`notification:notifications:read`	Users who set up alerts.	—	`dynatrace.automations`, `dynatrace.davis.problems`, `dynatrace.hub`	`GET /event-` (/platform/n `GET /event-` (/platform/n `GET /resour` (/platform/n +1 more
`notification:notifications:write`	Team leads and alert owners.	Readers.	`dynatrace.automations`, `dynatrace.davis.problems`, `dynatrace.hub`	`DELETE /eve` (/platform/n `DELETE /res` (/platform/n `DELETE /res` notification (/platform/n +4 more

Sources: APIs for latest Dynatrace (Dynatrace Developer) — Notification: *"This API was deprecated on 9 Jul 2025"*; IAM policy reference (DT docs).

10. Workflows (AutomationEngine)

IAM has no condition on a workflow's owner. Who sees and edits which workflow comes from ownership and visibility: a new workflow is private; the owner can make it public or transfer it to a group; admin mode reaches all of them.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`automation:workflows:read`	✅	Grants permission to read workflows	none	View workflows the caller can access (own, public, or group-owned).	Owner and visibility	Low	**Docs** — *"A public workflow is visible to every user with automation:workflows:* permissions."* (Workflow security)
`automation:workflows:write`	✅	Grants permission to write workflows	`automation:workflow-type` (IN, =)	Create and edit workflows; the docs Standard User policy limits it to `SIMPLE` (one task).	Owner and visibility; `automation:workflow-type`	Medium	**Docs** — *"A simple workflow is limited to only one task"* (Simple workflows)
`automation:workflows:run`	✅	Grants permission to execute workflows	none	Run workflows the caller can access.	Owner and visibility	Medium	**Docs** — *"Grants permission to execute workflows"* (IAM policy reference)
`automation:workflows:admin`	➕	Grant admin permissions for workflows.	none	Workflow admin mode — reach every workflow regardless of owner.	Environment-wide	High	**Docs** — *"Grant admin permissions for workflows."* (IAM policy reference)
`automation:calendars:read`	✅	Grants permission to read business calendars	none	Read business calendars used by workflow schedules.	Environment-wide	Low	**Docs** — *"Grants permission to read business calendars"* (IAM policy reference)
`automation:rules:read`	✅	Grants permission to read scheduling rules	none	Read scheduling rules (also used for maintenance-window recurrence).	Environment-wide	Low	**Tested** — `dynatrace.maintenancewindows` manifest: *"Read scheduling rules for maintenance window recurrence"* — validation tenant, 10/19/2026

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare i
`automation:workflows:read`	Users who need to see workflows.	—	`dynatrace.appshell`, `dynatrace.automations`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, `dynatrace.davis.workflow.actions`, +6 more	`GET /action-executions/{` (/platform/automation/v1) `GET /action-executions/{` (/platform/automation/v1) `GET /actions-sample-result/{actionIdentifier}` (/platform/automation/v1) +35 more

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare i
`automation:workflows:write`	Workflow authors; readers only with `workflow-type = "SIMPLE"`.	Unconditioned grants for readers.	`dynatrace.automations`, `dynatrace.clouds`, `dynatrace.security.threats.exploits`	`DELETE /trash/workflows/{` (/platform/automation/v1) `DELETE /workflows/{id}` (/platform/automation/v1) `PATCH /workflows/{id}` (/platform/automation/v1) +6 more
`automation:workflows:run`	Authors and operators.	—	`dynatrace.automations`, `dynatrace.security.investigator`	`POST /executions/{executionId}` (/platform/automation/v1) `POST /executions/{id}/can` (/platform/automation/v1) `POST /executions/{id}/pau` (/platform/automation/v1) +2 more
`automation:workflows:admin`	Admins.	Everyone else.	`dynatrace.automations`	no published operation declare
`automation:calendars:read`	Always.	—	`dynatrace.automations`, `dynatrace.dashboards`, `dynatrace.notebooks`	`GET /business-calendars` (/platform/automation/v1) `GET /business-calendars/{` (/platform/automation/v1) `GET /business-calendars/{` (/platform/automation/v1) +1 more
`automation:rules:read`	Always.	—	`dynatrace.automations`, `dynatrace.dashboards`, `dynatrace.maintenancewindows`, `dynatrace.notebooks`	`GET /scheduling-rules` (/platform/automation/v1) `GET /scheduling-rules/{id` (/platform/automation/v1) `GET /scheduling-rules/{id` (/platform/automation/v1) +1 more

> **Sources:** Workflow security (DT docs) — *"Transfer ownership to a group"*; Simple workflows (DT docs).

11. Davis analyzers and Copilot

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`davis:analyzers:read`	✅	Grants permission to view Davis analyzers	none	List analyzers and their documentation.	Environment-wide	Low	**Docs** — *"Grants permission to view Davis analyzers"* (IAM policy reference)
`davis:analyzers:execute`	✅	Grants permission to execute Davis analyzers	none	Run forecasting, anomaly and correlation analyzers on query results.	Environment-wide	Low	**Inference** — The analyzers take query results as input, so they reach only data the user can already read; not stated in the docs.
`davis-copilot:conversations:execute`	✅	Grants permission to interact with the Davis CoPilot conversational interface	none	Chat with Dynatrace Assist / Davis CoPilot.	Per user	Low	**Docs** — *"Grants permission to interact with the Davis CoPilot conversational interface"* (IAM policy reference)
`davis-copilot:nl2dql:execute`	✅	Grants permission to execute the Natural Language to DQL generative AI capability	none	Turn natural language into DQL.	Per user	Low	**Docs** — *"Grants permission to execute the Natural Language to DQL generative AI capability"* (IAM policy reference)
`davis-copilot:dql2nl:execute`	✅	Grants permission to execute the CoPilot skill 'Summarize DQL'	none	Explain a DQL query in plain language.	Per user	Low	**Docs** — *"Grants permission to execute the CoPilot skill 'Summarize DQL'"* (IAM policy reference)

| `davis-copilot:document-search:execute` | ✅ | Grants permission to execute the CoPilot skill 'Document Search' | none | Search documents with Copilot. | Per user | Low | **Docs** — *"Grants permission to execute the CoPilot skill 'Document Search'"* (IAM policy reference) |

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare it *(generated)*
`davis:analyzers:read`	Always.	—	`dynatrace.dashboards`, `dynatrace.davis.anomalydetection`, `dynatrace.davis.copilot`, `dynatrace.davis.workflow.actions`, `dynatrace.experience.vitals`, `dynatrace.notebooks`, +1 more	`GET /analyzers` (/platform/davis/analyzers/v `GET /analyzers/{analyzer-na` (/platform/davis/analyzers/v `GET /analyzers/{analyzer-name}/documentation` (/platform/davis/analyzers/v +2 more
`davis:analyzers:execute`	Users of Davis features in dashboards, notebooks and entity panels.	—	`dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, `dynatrace.davis.anomalydetection`, `dynatrace.davis.copilot`, `dynatrace.davis.problems`, +8 more	`GET /analyzers/{analyzer-name}:poll` (/platform/davis/analyzers/v `POST /analyzers/{analyzer-name}:cancel` (/platform/davis/analyzers/v `POST /analyzers/{analyzer-name}:execute` (/platform/davis/analyzers/v +1 more
`davis-copilot:conversations:execute`	When Copilot is enabled.	Where generative AI is not approved.	`dynatrace.appshell`, `dynatrace.clouds`, `dynatrace.dashboards`, `dynatrace.database.overview`, `dynatrace.davis.copilot`, `dynatrace.davis.problems`, +15 more	`POST /skills/conversations:feedba` (/platform/davis/copilot/v1) `POST /skills/conversations:messag` (/platform/davis/copilot/v1)
`davis-copilot:nl2dql:execute`	When Copilot is enabled.	Where generative AI is not approved.	`dynatrace.appshell`, `dynatrace.dashboards`, `dynatrace.davis.copilot`, `dynatrace.davis.workflow.actions`, `dynatrace.notebooks`	`POST /skills/nl2dql:feedbacl` (/platform/davis/copilot/v1) `POST /skills/nl2dql:generat` (/platform/davis/copilot/v1)
`davis-copilot:dql2nl:execute`	When Copilot is enabled.	Where generative AI is not approved.	`dynatrace.appshell`, `dynatrace.dashboards`, `dynatrace.davis.copilot`, `dynatrace.davis.workflow.actions`, `dynatrace.notebooks`	`POST /skills/dql2nl:explain` (/platform/davis/copilot/v1) `POST /skills/dql2nl:feedbacl` (/platform/davis/copilot/v1)
`davis-copilot:document-search:execute`	When Copilot is enabled.	Where generative AI is not approved.	`dynatrace.davis.copilot`, `dynatrace.davis.problems`, `dynatrace.davis.workflow.actions`	`POST /skills/document-search:execute` (/platform/davis/copilot/v1)

> **Sources:** IAM policy reference (DT docs).

12. OpenPipeline

The two rows share a prefix and nothing else: one reads pipeline configuration, the other writes Davis events.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis

| `openpipeline:configurations:read` | ✅ | Grants permission to read the OpenPipeline configuration | none | Read pipeline configuration (processing, masking, routing, security-context rules); also drives DQL-matcher autocomplete. | Environment-wide | Low | **Tested** — `dynatrace.observability.settings` manifest: *"required for DQL matcher autocomplete"* — validation tenant, 10/19/2026 |
| `openpipeline:events.davis:ingest` | ➕ | Grants permission to ingest davis events into OpenPipeline | `openpipeline:event-type` (=, IN, startsWith, MATCH)
`openpipeline:event-provider` (=, IN, startsWith, MATCH) | Write Davis events. With provider `PROBLEM_APP` and type `CUSTOM_ANNOTATION`, the Problems app uses it for comments and closing problems. | Environment-wide; conditioned by provider and type | Low to medium | **Tested** — `dynatrace.davis.problems` manifest: *"Write problem comments and close problems"* (not in the docs Standard User policy) — validation tenant, 10/19/2026 |

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare i *(generated)*
`openpipeline:configurations:read`	Pipeline owners; optional for others.	Readers, as a least-privilege choice — it reveals how data separation is built.	`dynatrace.automations`, `dynatrace.observability.settings`, `dynatrace.openpipeline`	`GET /configurations` (`/platform/openpipeline/v`
`GET /technologies` (`/platform/openpipeline/v` *(early adopter)*
`GET /technologies/{id}/proces` (`/platform/openpipeline/v` *(early adopter)*
+6 more
`openpipeline:events.davis:ingest`	Users who may comment on or close problems.	Readers, if commenting is a write you don't allow. Not needed to view problems.	`dynatrace.davis.problems`	no published operation declare

> **Sources:** IAM policy reference (DT docs).

13. Synthetic monitoring

Monitors take `synthetic:dt.security_context`; locations take no condition at all. To keep users off public locations you don't use, change those locations' status for the environment: `DISABLED` stops new assignments and leaves assigned monitors running; `HIDDEN` removes the location from the UI and is allowed only when no monitor uses it. Holding `synthetic:locations:write` decides who can do that.

Dynatrace *"recommends migrating to the Synthetic platform permissions and the Synthetic Platform API"* (`/platform/synthetic/v1`, platform or OAuth token). Two things to know before you do:

- **The platform API is Early Adopter.** *"All Synthetic Platform API endpoints are currently in Early Adopter maturity"* — they *"can change in non-compatible ways"*. The validation tenant's own spec marks all 20 operations the same way (10/19/2026). The classic Synthetic API v2 keeps working.
- **Security-context scoping needs the platform permissions.** Users who hold only classic permissions see every monitor; boundaries don't restrict them.

Two facts from the same page that the tables can't show:

- **The Private locations tab needs unscoped data reads.** It requires `storage:smartscape:read` and `storage:metrics:read` with no security-context condition, plus `synthetic:locations:read` (or `:write` to manage) — *"These permissions must be applied tenant-wide for the tab to be accessible."*
- **The two docs pages disagree on *Standard User*.** The Synthetic access-control page (updated 10/13/2026) lists `synthetic:monitors:read` and `synthetic:locations:read` in *Standard User*; the default-policies page this appendix generates from (modified 10/08/2026) does not, so both rows show ➕. Check the policy in your account — the newer page is likely current.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`synthetic:monitors:read`	➕	Grants permission to read definitions of synthetic monitors	`synthetic:dt.security_context` (IN, =, startsWith, MATCH)	View synthetic monitor definitions.	Per security context	Low	**Docs** — *"Grants permission to read definitions of synthetic monitors"* (IAM policy reference)
`synthetic:monitors:write`	➕	Grants permission to create synthetic monitors	`synthetic:dt.security_context` (IN, =, startsWith, MATCH)	Create, change and delete monitors.	Per security context	Medium	**Tested** — Synthetic API: `POST` , `PUT` and `DELETE` `/monitors` all declare it — the reference mentions only create — validation tenant, 10/19/2026
`synthetic:monitors:execute`	➕	Grants permission to schedule on-demand executions	`synthetic:dt.security_context` (IN, =, startsWith, MATCH)	Trigger on-demand monitor executions.	Per security context	Low	**Docs** — *"Grants permission to schedule on-demand executions"* (IAM policy reference)
`synthetic:locations:read`	➕	Grants permission to read definitions of locations	none	List public and private synthetic locations and nodes, and download private-location deployment YAML. Has no conditions; to keep users off public locations, change those locations' status (needs `synthetic:locations:write`).	Environment-wide, no conditions	Low	**Docs** — *"The response includes all matching public and private locations available for your environment."* (GET locations (platform API))
`synthetic:locations:write`	➕	Grants permission to create locations	none	Create, update and delete private locations; set a public location to `ENABLED` , `DISABLED` (can't be assigned; assigned monitors keep running) or `HIDDEN` (only when no monitor uses it); switch all public locations off at once.	Environment-wide, no conditions	High	**Docs** — *"for public locations you can only change the location status"* (PUT synthetic location (platform API))
`synthetic:environment-configuration:read`	➕	Grants permission to read environment configuration	none	Read the environment-wide Synthetic settings, such as browser-monitor and step timeouts.	Environment-wide	Low	**Docs** — *"the synthetic-related parameters defined for the whole tenant"* (GET configuration (platform API))
`synthetic:environment-configuration:write`	➕	Grants permission to edit environment configuration	none	Change those settings for every monitor in the environment (`bmMonitorTimeout` , `bmStepTimeout`).	Environment-wide	Medium	**Docs** — *"change the synthetic-related parameters defined for the whole tenant"* (PUT configuration (platform API))

Permission	**Allow when**	**Avoid when**	**Built-in apps that request it *(generated)***	**API operations it *(generated)***
`synthetic:monitors:read`	Readers — and add a `synthetic:dt.security_context` line to the boundary.	A boundary with no `synthetic:` line, which leaves every monitor visible.	`dynatrace.dashboards` , `dynatrace.experience.vitals` , `dynatrace.notebooks` , `dynatrace.synthetic`	`GET /monitor` `(/platform/sy` *(early adopter)* `GET /monitor` `id}` `(/platform/sy` *(early adopter)*
`synthetic:monitors:write`	Team leads, scoped.	Readers.	`dynatrace.experience.vitals` , `dynatrace.synthetic`	`DELETE` `/monitors/{mo` `(/platform/sy` *(early adopter)* `POST /monito` `(/platform/sy` *(early adopter)* `PUT /monitor` `id}` `(/platform/sy` *(early adopter)*
`synthetic:monitors:execute`	Team leads, scoped.	Readers.	`dynatrace.synthetic`	`POST /execut` `(/platform/sy` *(early adopter)* `POST` `/executions/{` `id}` `(/platform/sy` *(early adopter)*

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations it *(generated)*
`synthetic:locations:read`	Anyone who views or creates monitors.	—	`dynatrace.dashboards`, `dynatrace.notebooks`, `dynatrace.synthetic`	`GET /locatio` (`/platform/sy` *(early adopter)* `GET /locatio` `adapter/yaml` (`/platform/sy` *(early adopter)* `GET /locatio` (`/platform/sy` *(early adopter)* +4 more
`synthetic:locations:write`	Synthetic admins who decide where monitors may run.	Monitor authors — it changes where everyone's monitors can run.	`dynatrace.synthetic`	`DELETE /locations/{` (`/platform/sy` *(early adopter)* `POST /locati` (`/platform/sy` *(early adopter)* `PUT /locatio` (`/platform/sy` *(early adopter)* +1 more
`synthetic:environment-configuration:read`	Synthetic admins and monitor authors.	—	`dynatrace.dashboards`, `dynatrace.notebooks`, `dynatrace.synthetic`	`GET /configu` (`/platform/sy` *(early adopter)*
`synthetic:environment-configuration:write`	Synthetic admins.	Everyone else.	none declare it	`PUT /configu` (`/platform/sy` *(early adopter)*

> **Sources:** Access control — Synthetic (DT docs) — as quoted; PUT synthetic location — platform API (DT docs) — *"for public locations you can only change the location status"*; PUT a synthetic location (DT docs); IAM policy reference (DT docs).

14. Credential vault

IAM decides whether a user reaches the vault; each credential then has its own owner, sharing mode (owner only, shared, public) and scopes.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`credential-vault:entries:read`	➕	Grants permission to read entries from the credential vault	none	List and read vault entries the caller may see; each credential also has its own owner, sharing and scope.	Per credential (owner-only, shared, public)	High	**Unknown** — The vault page describes access in classic-role terms (View environment); how that maps to `credential-vault:*` is undocumented.
`credential-vault:entries:write`	➕	Grants permission to write entries to the credential vault	none	Create and change vault entries.	Per credential	High	**Tested** — Credential vault API: `POST /entries`, `PUT` and `DELETE /entries/{entry-id}` declare it — validation tenant, 10/19/2026
`credential-vault:entries:admin`	➕	Grants permission to overwrite entries in the credential vault	none	Overwrite vault entries — by our reading, including entries other users own.	Environment-wide	High	**Inference** — The reference says only *"overwrite entries"*; that it bypasses ownership is read from the vault page's owner rule, which is written in classic-role terms. No API operation declares it.

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare it *(generated)*
`credential-vault:entries:read`	Monitor and connector authors.	Readers.	`dynatrace.dashboards`, `dynatrace.notebooks`, `dynatrace.synthetic`	`GET /entries (/platform/credential-vault/v1)` *(early adopter)* `GET /entries/{entry-id} (/platform/credential-vault/v1)` *(early adopter)*
`credential-vault:entries:write`	Monitor and connector authors.	Readers.	`dynatrace.synthetic`	`DELETE /entries/{entry-id} (/platform/credential-vault/v1)` *(early adopter)* `POST /entries (/platform/credential-vault/v1)` *(early adopter)* `PUT /entries/{entry-id} (/platform/credential-vault/v1)` *(early adopter)*
`credential-vault:entries:admin`	Admins.	Everyone else.	none declare it	no published operation declares it

> **Sources:** Credential vault (DT docs); IAM policy reference (DT docs).

15. Security

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`vulnerability-service:vulnerabilities:read`	✅	Allows viewing vulnerabilities	none	View vulnerabilities and Davis security recommendations.	Environment-wide	Medium	**Docs** — *"Allows viewing vulnerabilities"* (IAM policy reference)
`security-intelligence:enrichments:run`	✅	Allows execution of enrichments and discovery of integration apps.	none	Run threat-intelligence enrichments (e.g. on an IP) and discover integration apps.	Environment-wide	Medium	**Docs** — *"Allows execution of enrichments and discovery of integration apps."* (IAM policy reference)
`environment:roles:view-security-problems`	✅	Grants user the View security problems permission.	`environment:management-zone` (IN, startsWith, NOT startsWith, =, !=, MATCH)	Classic *View security problems* role.	Per management zone	Medium	**Docs** — *"Grants user the View security problems permission."* (IAM policy reference)

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare it *(generated)*
`vulnerability-service:vulnerabilities:read`	Security teams.	General readers.	`dynatrace.security.vulnerabilities`	`GET /davis-security-recommendations (/platform/vulnerabilities/v1)` `GET /vulnerabilities (/platform/vulnerabilities/v1)` `GET /vulnerabilities/{id} (/platform/vulnerabilities/v1)` +7 more
`security-intelligence:enrichments:run`	Security teams.	General readers.	`dynatrace.security.enrichment`, `dynatrace.security.investigator`, `dynatrace.security.threats.exploits`	no published operation declares it
`environment:roles:view-security-problems`	Security teams on classic pages.	General readers.	none declare it	no published operation declares it

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare it *(generated)*

> **Sources:** IAM policy reference (DT docs).

16. Live Debugger

Breakpoints capture runtime state, and the snapshots land in `application.snapshots` (see Grail data). Grant the two together, or neither.

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`dev-obs:breakpoints:set`	✅	Grants permission to set breakpoint using DevObs live debugger	`dev-obs:k8s.namespace.name` (=, IN, startsWith, !=, NOT IN, NOT startsWith) `dev-obs:dt.entity.process_group` (=, IN, startsWith, !=, NOT IN, NOT startsWith) `dev-obs:dt.process_group.detected_name` (=, IN, startsWith, !=, NOT IN, NOT startsWith) `dev-obs:k8s.cluster.name` (=, IN, startsWith, !=, NOT IN, NOT startsWith) `dev-obs:host.group` (=, IN, startsWith, !=, NOT IN, NOT startsWith) `dev-obs:host.name` (=, IN, startsWith, !=, NOT IN, NOT startsWith)	Place non-breaking breakpoints that capture runtime state.	Per process group, host or namespace	High	**Docs** — *"variable values"* (Live Debugger)
`dev-obs:breakpoints:manage`	➕	Grants permission to manage breakpoints set in DevObs live debugger	none	View and disable breakpoints placed by any user.	Environment-wide	Medium	**Tested** — `dynatrace.devobs.debugger` manifest: *"viewing and disabling breakpoints placed by any user"* — validation tenant, 10/19/2026

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare it *(generated)*
`dev-obs:breakpoints:set`	Developers, scoped to their workloads.	Readers, and anywhere runtime data is sensitive.	`dynatrace.devobs.debugger`	no published operation declares it
`dev-obs:breakpoints:manage`	Admins.	Everyone else.	`dynatrace.devobs.debugger`	no published operation declares it

> **Sources:** Live Debugger (DT docs).

17. Classic roles

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`environment:roles:viewer`	✅	Grants user the Access environment permission.	`environment:management-zone` (IN, startsWith, NOT startsWith, =, !=, MATCH)	Classic *Access environment*: opens classic pages and APIs that check this role.	Per management zone	Medium	**Docs** — *"Grants user the Access environment permission."* (IAM policy reference)

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations that declare it *(generated)*
`environment:roles:viewer`	While users still work in classic pages; scope with `environment:management-zone`.	Unscoped grants.	none declare it	no published operation declares it

> **Sources:** IAM policy reference (DT docs).

18. Other services

Permission	Std User	Reference says *(generated)*	Conditions *(generated)*	What it really grants	Scope	Risk	Basis
`geolocation:locations:lookup`	✅	Grants permission to lookup geolocations for IP adresses.	none	Resolve IP addresses to countries, regions and cities.	Environment-wide	Low	**Tested** — `dynatrace.experience.vitals` manifest: *"Required for looking up countries, regions, and cities"* — validation tenant, 10/19/2026
`slo:slos:read`	✅	Grants permission to read Service-Level Objectives	none	Read service-level objectives.	Environment-wide	Low	**Docs** — *"Grants permission to read Service-Level Objectives"* (IAM policy reference)
`slo:objective-templates:read`	✅	Grants permission to read Service-Level Objectives Templates	none	Read SLO templates.	Environment-wide	Low	**Docs** — *"Grants permission to read Service-Level Objectives Templates"* (IAM policy reference)
`insights:opportunities:read`	✅	Grants permission to read data from the Opportunity Insights API	none	Read Opportunity Insights data.	Environment-wide	Low	**Docs** — *"Grants permission to read data from the Opportunity Insights API"* (IAM policy reference)
`insights:moments:read`	✅	Grants permission to query value moments and related data	none	Read value moments.	Environment-wide	Low	**Docs** — *"Grants permission to query value moments and related data"* (IAM policy reference)
`extensions:definitions:read`	✅	Grants permission to read extension and environment configurations	`extensions:extension-name` (IN, NOT IN, startsWith, NOT startsWith, !=, =)	Read extension definitions and their environment configuration.	Per extension name	Low	**Docs** — *"Grants permission to read extension and environment configurations"* (IAM policy reference)
`business-analytics:business-flows:read`	✅	Grants permission to read business-flows	none	Read business flows; the Problems app reads affected flows with it.	Environment-wide	Low	**Tested** — `dynatrace.davis.problems` manifest: *"Read evaluation of affected Business Flows"* — validation tenant, 10/19/2026
`deobfuscation:symbol-files:read`	➕	Grants permission to retrieve information regarding Symbol files.	none	Read symbol-file information used to de-obfuscate RUM stack traces.	Environment-wide	Low	**Docs** — *"Grants permission to retrieve information regarding Symbol files."* (IAM policy reference)
`support-and-services:support:read`	➕	Grants permission to view support cases and chats in the Support and Services app. With this permission, a user sees cases they created and cases they were added to as a collaborator.	none	See support cases the caller created or was added to.	Per user	Low	**Docs** — *"a user sees cases they created and cases they were added to"* (IAM policy reference)
`iam:effective-permissions:read`	➕	Grants permission to read effective permissions	`iam-param:entity-type` (=) `iam-param:entity-id` (=, IN)	Account-level effective-permissions API for any user or group — an OAuth client scope, not the self-check.	Account-wide (`iam-param:entity-type` / `entity-id`)	Medium	**Docs** — *"Grants permission to read effective permissions"* (IAM policy reference)

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations *(generated)*
`geolocation:locations:lookup`	With RUM data permissions.	—	`dynatrace.experience.vitals` , `dynatrace.synthetic`	no published ope
`slo:slos:read`	When the SLO app or SLO tiles are used.	—	`dynatrace.clouds` , `dynatrace.dashboards` , `dynatrace.davis.problems` , `dynatrace.infraops` , `dynatrace.kubernetes` , `dynatrace.service.level.objectives`	`GET /objectiv` `(/platform/slo` `GET /objectiv` `(/platform/slo` `GET /slos (/p` +4 more
`slo:objective-templates:read`	With slo:slos:read.	—	`dynatrace.infraops` , `dynatrace.kubernetes` , `dynatrace.service.level.objectives`	`GET /objectiv` `(/platform/slo` `GET /objectiv` `(/platform/slo`
`insights:opportunities:read`	When the app is used.	When it isn't.	none declare it	no published ope

Permission	Allow when	Avoid when	Built-in apps that request it *(generated)*	API operations *(generated)*
`insights:moments:read`	When Business Insights is used.	When it isn't.	none declare it	no published ope
`extensions:definitions:read`	Always; several infrastructure apps use it.	—	`dynatrace.clouds`, `dynatrace.database.overview`, `dynatrace.extensions.manager`, `dynatrace.fleet.management`, `dynatrace.hub`, `dynatrace.infraops`, +2 more	`GET /extensio (/platform/ext` `GET /extensio (/platform/ext` `GET /extensio name}/environm (/platform/ext` +6 more
`business-analytics:business-flows:read`	When Business Flow is used.	When it isn't.	`dynatrace.davis.problems`	no published ope
`deobfuscation:symbol-files:read`	Frontend and mobile teams.	Readers without RUM.	`dynatrace.experience.vitals`	`GET /symbol-f (/platform/rum` `GET /symbol-f (/platform/rum` `GET /symbol-f download (/platform/rum` +2 more
`support-and-services:support:read`	Your call — users see only their own cases.	—	none declare it	no published ope
`iam:effective-permissions:read`	Admins and access-review automation.	Confusing it with `platform-management:effective-permissions:resolve`.	none declare it	no published ope

> **Sources:** IAM policy reference (DT docs).

19. Persona Matrix

Five personas, as a starting point for **IAM-11**'s workshop — adjust them to your organization.

Persona	Does
Reader	Opens dashboards, notebooks and allowed apps for their team's data
Team lead	Reader, plus curates team content: shares, segments, team settings and monitors
Workflow author	Builds and runs workflows, connections and notifications
Security	Works with vulnerabilities, security events and threat enrichment
Admin	Runs the environment

Mark	Meaning
✅	Grant
🔶	Grant, scoped by a condition or the boundary

Mark	Meaning
⚪	Optional — grant when the feature is used
—	Don't grant

Permission	Reader	Team lead	Workflow author	Security	Admin
App state					
`state:app-states:read`	✅	✅	✅	✅	✅
`state:app-states:write`	—	⚪	⚪	⚪	✅
`state:app-states:delete`	—	—	—	—	✅
`state:user-app-states:read`	✅	✅	✅	✅	✅
`state:user-app-states:write`	✅	✅	✅	✅	✅
`state:user-app-states:delete`	✅	✅	✅	✅	✅
`state-management:user-app-states:delete`	⚪	✅	✅	✅	✅
`state-management:user-app-states:delete-all`	—	—	—	—	✅
`state-management:app-states:delete`	—	—	—	—	✅
Documents					
`document:documents:read`	✅	✅	✅	✅	✅
`document:documents:write`	⚪	✅	✅	✅	✅
`document:documents:delete`	⚪	✅	✅	✅	✅
`document:documents:admin`	—	—	—	—	✅
`document:environment-shares:read`	✅	✅	✅	✅	✅
`document:environment-shares:write`	—	✅	✅	✅	✅
`document:environment-shares:claim`	✅	✅	✅	✅	✅
`document:environment-shares:delete`	—	✅	✅	✅	✅
`document:direct-shares:read`	✅	✅	✅	✅	✅
`document:direct-shares:write`	⚪	✅	✅	✅	✅
`document:direct-shares:delete`	⚪	✅	✅	✅	✅
`document:trash.documents:read`	✅	✅	✅	✅	✅
`document:trash.documents:restore`	✅	✅	✅	✅	✅
`document:trash.documents:delete`	—	✅	✅	✅	✅
Apps, functions and the UI shell					
`app-engine:apps:run`	✅	✅	✅	✅	✅
`app-engine:functions:run`	✅	✅	✅	✅	✅
`app-engine:edge-connects:read`	—	⚪	✅	—	✅
`unified-analysis:screen-definition:read`	✅	✅	✅	✅	✅
`hub:catalog:read`	✅	✅	✅	✅	✅
`platform-management:effective-permissions:resolve`	✅	✅	✅	✅	✅
`platform-management:environments:read`	✅	✅	✅	✅	✅
Grail definitions and segments					
`storage:bucket-definitions:read`	✅	✅	✅	✅	✅
`storage:fieldset-definitions:read`	✅	✅	✅	✅	✅
`storage:filter-segments:read`	✅	✅	✅	✅	✅
`storage:filter-segments:write`	—	✅	⚪	⚪	✅
`storage:filter-segments:delete`	—	✅	⚪	⚪	✅
`storage:filter-segments:share`	—	✅	—	⚪	✅
`storage:filter-segments:admin`	—	—	—	—	✅
Grail data (record tables)					
`storage:buckets:read`	✅	✅	✅	✅	✅
`storage:logs:read`	🔶	🔶	🔶	✅	✅
`storage:spans:read`	🔶	🔶	🔶	✅	✅
`storage:metrics:read`	🔶	🔶	🔶	✅	✅
`storage:events:read`	🔶	🔶	🔶	✅	✅
`storage:bizevents:read`	⚪	🔶	🔶	⚪	✅
`storage:entities:read`	🔶	🔶	🔶	✅	✅
`storage:smartscape:read`	🔶	🔶	🔶	✅	✅
`storage:system:read`	—	—	⚪	✅	

`storage:files:read`	🔶	🔶	🔶	🔶	✅
`storage:user.events:read`	⚪	🔶	⚪	⚪	✅
`storage:user.sessions:read`	⚪	🔶	⚪	⚪	✅
`storage:security.events:read`	—	—	—	✅	✅
`storage:application.snapshots:read`	—	—	—	⚪	✅
Settings and app settings					
`settings:objects:read`	🔶	🔶	🔶	🔶	✅
`settings:schemas:read`	✅	✅	✅	✅	✅
`settings:objects:write`	🔶	🔶	🔶	🔶	✅
`settings:objects:admin`	—	—	—	—	✅
`app-settings:objects:read`	🔶	🔶	✅	🔶	✅
`app-settings:objects:write`	—	⚪	✅	⚪	✅
`app-settings:objects:admin`	—	—	—	—	✅
Email and notifications					
`email:emails:send`	—	⚪	✅	⚪	✅
`notification:self-notifications:read`	⚪	✅	✅	✅	✅
`notification:self-notifications:write`	⚪	✅	✅	✅	✅
`notification:notifications:read`	⚪	✅	✅	✅	✅
`notification:notifications:write`	—	✅	✅	⚪	✅
Workflows (AutomationEngine)					
`automation:workflows:read`	⚪	✅	✅	✅	✅
`automation:workflows:write`	—	🔶	✅	⚪	✅
`automation:workflows:run`	⚪	✅	✅	✅	✅
`automation:workflows:admin`	—	—	—	—	✅
`automation:calendars:read`	✅	✅	✅	✅	✅
`automation:rules:read`	✅	✅	✅	✅	✅
Davis analyzers and Copilot					
`davis:analyzers:read`	✅	✅	✅	✅	✅
`davis:analyzers:execute`	✅	✅	✅	✅	✅
`davis-copilot:conversations:execute`	⚪	⚪	⚪	⚪	✅
`davis-copilot:nl2dql:execute`	⚪	⚪	⚪	⚪	✅
`davis-copilot:dql2nl:execute`	⚪	⚪	⚪	⚪	✅
`davis-copilot:document-search:execute`	⚪	⚪	⚪	⚪	✅
OpenPipeline					
`openpipeline:configurations:read`	—	⚪	⚪	⚪	✅
`openpipeline:events.davis:ingest`	⚪	✅	⚪	✅	✅
Synthetic monitoring					
`synthetic:monitors:read`	🔶	🔶	🔶	✅	✅
`synthetic:monitors:write`	—	🔶	⚪	⚪	✅
`synthetic:monitors:execute`	—	🔶	🔶	⚪	✅
`synthetic:locations:read`	✅	✅	✅	✅	✅
`synthetic:locations:write`	—	—	—	—	✅
`synthetic:environment-configuration:read`	—	⚪	⚪	—	✅
`synthetic:environment-configuration:write`	—	—	—	—	✅
Credential vault					
`credential-vault:entries:read`	—	⚪	✅	⚪	✅
`credential-vault:entries:write`	—	⚪	✅	⚪	✅
`credential-vault:entries:admin`	—	—	—	—	✅
Security					
`vulnerability-service:vulnerabilities:read`	—	⚪	—	✅	✅
`security-intelligence:enrichments:run`	—	—	—	✅	✅
`environment:roles:view-security-problems`	—	—	—	✅	✅
Live Debugger					
`dev-obs:breakpoints:set`	—	⚪	—	⚪	✅
`dev-obs:breakpoints:manage`	—	—	—	—	✅
Classic roles					
`environment:roles:viewer`	🔶	🔶	🔶	✅	✅
Other services					
`geolocation:locations:lookup`	⚪	✅	⚪	✅	✅

`slo:slos:read`	⚪	✅	✅	✅	✅
`slo:objective-templates:read`	⚪	✅	✅	✅	✅
`insights:opportunities:read`	⚪	⚪	—	—	✅
`insights:moments:read`	⚪	⚪	—	—	✅
`extensions:definitions:read`	✅	✅	✅	✅	✅
`business-analytics:business-flows:read`	⚪	⚪	—	—	✅
`deobfuscation:symbol-files:read`	—	⚪	—	—	✅
`support-and-services:support:read`	⚪	✅	⚪	✅	✅
`iam:effective-permissions:read`	—	—	—	—	✅

20. Open Questions

Every row whose Basis is **Inference** or **Unknown**, generated from the data. Settle one with a test user (§ 2.4), then change its Basis to **Tested** with the date.

Permission	Basis	What would settle it
`state:app-states:write`	**Inference** — Follows from app states being shared (state service page); 18 built-in apps declare it, e.g. `dynatrace.discovery.coverage` *"write mute state"*.	A test user without it (§ 2.4), or a docs page.
`state:app-states:delete`	**Inference** — Same reasoning as write; the state service page shows a delete that *"completely reset[s] the app's states"*.	A test user without it (§ 2.4), or a docs page.
`document:documents:write`	**Inference** — Ownership limits it in practice; no IAM condition exists, so scope comes from document ownership and shares.	A test user without it (§ 2.4), or a docs page.
`app-engine:edge-connects:read`	**Unknown** — Whether apps or workflows that send traffic through an EdgeConnect fail without it is not documented; `dynatrace.dashboards` and `dynatrace.notebooks` declare it *"via functions"*.	A test user without it (§ 2.4), or a docs page.
`notification:notifications:write`	**Unknown** — Whether a user can change notifications created by others is not documented; the API has no owner filter in its declared scopes.	A test user without it (§ 2.4), or a docs page.
`davis:analyzers:execute`	**Inference** — The analyzers take query results as input, so they reach only data the user can already read; not stated in the docs.	A test user without it (§ 2.4), or a docs page.
`credential-vault:entries:read`	**Unknown** — The vault page describes access in classic-role terms (View environment); how that maps to `credential-vault:*` is undocumented.	A test user without it (§ 2.4), or a docs page.
`credential-vault:entries:admin`	**Inference** — The reference says only *"overwrite entries"*; that it bypasses ownership is read from the vault page's owner rule, which is written in classic-role terms. No API operation declares it.	A test user without it (§ 2.4), or a docs page.

21. Regenerating the Fact Columns

The generated columns come from three data files in the generator repository's `topics/iam/docs/data/`:

File	Holds	Refreshed by
`iam-permissions.json`	Every service, permission, description, condition and operator; the parsed *Standard User* policy	`--fetch` (docs pages)
`iam-tenant-evidence.json`	Built-in app manifest scopes and platform API operation scopes	`--tenant-context <dtctl context>`
`iam-97-judgements.json`	The judgement columns and Basis, by hand	Editing

```
python3 scripts/generate_iam_permission_reference.py --fetch --tenant-context &lt;ctx&gt; --render
python3 scripts/generate_iam_permission_reference.py --check
```

`--check` fails when the notebook is stale, a row lacks a Basis, a Docs quote is 15 words or longer, or a judgement names a permission the reference no longer lists — so a renamed or removed permission surfaces as an error instead of a silently wrong row.

22. References

- IAM policy statements reference (DT docs)
- Default policies (DT docs)
- Policy boundaries (DT docs)
- Permissions in Grail (DT docs)
- Upgrade security policies for scoped settings (DT docs)
- Workflow security (DT docs)
- Simple workflows (DT docs)
- Credential vault (DT docs)
- Live Debugger (DT docs)
- PUT a synthetic location (DT docs)
- Manage users, groups, and permissions (DT docs)
- Upgrade role-based permissions to IAM (DT docs)
- Store app and user states (Dynatrace Developer)
- Store app settings (Dynatrace Developer)
- Add permission scopes (Dynatrace Developer)
- App Engine registry SDK (Dynatrace Developer)
- Platform management SDK (Dynatrace Developer)

Related notebooks

- **IAM-04** — policy syntax and the services table
- **IAM-05** — boundary design
- **IAM-09** — troubleshooting access
- **IAM-11** — persona workshop
- **MZ2POL-12** — IAM reference architecture

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.